

Friday named the three ways to pass a parameter at the end of a heavy class: by value, by reference, and by const reference, with the in / out / in-out vocabulary attached.

Today: go back and make the direction the point. The three ways are syntax. The question behind them is what a parameter is for, and that question is the one thing from last week the current assignment is actually testing.

PART

1. Which way is the data going?
2. Live: three functions, one variable
3. Reading a parameter list

Part 1. Which way is the data going?

Three jobs a parameter can have, and the declaration should say which:

| | What it is for | How it is written | |--| | **in** | the function reads it | by value, or `const` & if it is big | | **out** | the function fills it in | by reference | | **in/out** | the function reads and changes it | by reference |

Talk to your neighbor. A function that swaps two numbers. Which kind are its parameters, and what does that force the declaration to look like?

Part 2. Live: three functions, one variable

Write all three and run them on the same variable, printing the address inside the caller and inside each function.

```
void byValue(int n)          { n = 99; }  
void byRef  (int &n)         { n = 99; }  
void byConst(const int &n)   { /* n = 99; would not compile */ }
```

Then uncomment the assignment in the third and let the compiler answer:

```
error: assignment of read-only reference 'n'
```

const is not documentation. It is a promise the compiler enforces.

Part 3. Reading a parameter list

Put an unfamiliar signature on the board and read it aloud together, one parameter at a time: what goes in, what comes back, what is promised not to change. This is a skill they need on every page from here on, and it is worth ten minutes of practice while nothing else is at stake.